

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Курсовая работа

По дисциплине

“Информационные системы и базы данных”

Этап 3

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р33101

Преподаватель:
Гаврилов Антон Валерьевич

Санкт-Петербург, 2023г

Задание

Этап 3.

Реализовать даталогическую модель в реляционной СУБД PostgreSQL:

- Создать необходимые объекты базы данных.
- Заполнить созданные таблицы тестовыми данными.
- Сделать скрипты для:
 - создания/удаления объектов базы данных;
 - заполнения/удаления созданных таблиц.
- Обеспечить целостность данных при помощи средств языка DDL.
- Добавить в базу данных триггеры для обеспечения комплексных ограничений целостности.
- Реализовать функции и процедуры на основе описания бизнес-процессов (из этапа №1).
- Произвести анализ использования созданной базы данных:
 - выявить наиболее часто используемые запросы к объектам базы данных;
 - результаты представить в виде текстового описания.
- Создать индексы и доказать, что они полезны для вашей базы данных:
 - доказательство должно быть приведено в виде текстового описания.

Ход работы

Описание предметной области

Для стартапа требуется разработать информационную систему. Бизнес-идеей стартапа является сервис по вендингу печати. Для реализации бизнес-идеи необходимо веб-приложение, через которое клиент сможет сделать заказ на печать и оплатить его. После оплаты заказа у клиента должна быть возможность забрать свой заказ в вендинговом аппарате.

Веб-приложение должно обладать возможностью авторизации и регистрации с предоставлением разных привилегий доступа. Привилегии должны иметь разграничение по ролям. Роли, которыми обладают пользователи будут указывать на привилегии доступа к эндпоинтам приложения и дополнительную информацию характерную для каждой роли. Также оно должно реализовывать личный кабинет пользователя со счётом, который пользователь сможет пополнить и оформить через него заказ. Личный счет должен иметь привязку к аккаунту пользователя, но при этом поскольку предполагается возможность наличия нескольких счетов для каждого аккаунта, он должен представлять из себя отдельную сущность. Взаимодействие с личным счетом будет осуществляться в виде списания со счета и пополнения. Списания будут происходить при оформлении заказа, а пополнения при оформлении возврата заказа, либо пополнении счета через платежный сервис онлайн-кассы от стороннего разработчика на стороннем сайте (онлайн-касса жестко регламентируется законом, по этой причине реализация собственной не имеет смысла для этого проекта). Для симуляции пополнений счета можно сделать отдельную форму через которую вводить сумму пополнения счета.

Оформление заказов должно осуществляться при помощи заполнения формы. Есть 2 типа заказов — заказ на печать и заказ на сканирование. Для каждого типа заказа своя форма с разными полями для оформления. При оформлении каждого заказа указывается место вендинговой точки, где клиент хочет забрать заказ. Для заказа на печать должна быть возможность загрузить файлы, которые клиент хочет распечатать, при этом должна быть

возможность выбрать тип печати (цветная или черно-белая). Для заказа на сканирование должна быть возможность указать количество страниц, которые клиент хочет отсканировать. После заполнения формы заказа можно будет добавить заказ в корзину, чтобы оплатить его в ней.

Корзина представляет из себя список неоплаченных заказов, с возможностью их оплатить или удалить.

После оплаты заказа на его основе должны формироваться задания и передаваться нужному вендинговому аппарату. Под заданиями понимается набор информации, достаточный для того, чтобы частично или полностью обслужить заказа. При этом для каждого заказа должен формироваться такой набор, заданий, чтобы он был полностью выполнен. Также каждому оплаченному заказу присваивается номер, введя который в вендинговом аппарате, клиент сможет забрать свой заказ. Соответственно клиент должен иметь возможность увидеть номер нужного ему заказа. Также должна быть возможность удалить заказ, если он еще не был выполнен, при удалении оплаченного заказа средства возвращаются на счет клиента. Каждая вендинговая точка может иметь несколько вендинговых аппаратов, с разной комплектацией по функционалу. То есть различные вендинговые аппараты могут иметь разные виды печати (цветную, черно-белую или все сразу), и наличие либо отсутствие функции сканирования.

Пользователь должен иметь возможность просмотреть список загруженных файлов. Файлы в этом списке делятся на 2 категории — те которые пользователь загрузил при оформлении заказа на печать, и те которые были получены от вендингового аппарата после того как клиент отсканировал нужные ему документы.

Информация о состоянии сети вендинговых аппаратов должна храниться в базе данных и регулярно обновляться. Такой информацией является, та которая обновляется при выполнении заказов или обслуживании аппаратов — запас бумаги в аппарате, чернил. Либо та информация, которая обновляется периодически — статус работы вендингового аппарата (работает, сломан, временно недоступен и т. д.).

Исходный код

Исходный код на github - <https://github.com/stoneshik/is-and-bd-coursework-3>

Анализ созданной базы данных

Для наиболее часто используемых запросов к объектам, где обращаются не к первичному ключу, а к обычному полю были созданы следующие индексы

```
CREATE INDEX user_email_index_hash ON users USING hash(user_email);
CREATE INDEX user_login_index_hash ON users USING hash(user_login);

CREATE INDEX order_type_index_hash ON orders USING hash(order_type);
CREATE INDEX order_status_index_hash ON orders USING hash(order_status);

CREATE INDEX vending_point_schedule_day_week_index_hash ON vending_point_schedules
    USING hash(vending_point_schedule_day_week);
CREATE INDEX vending_point_unusual_schedule_date_index_hash ON vending_point_unusual_schedules
    USING hash(vending_point_unusual_schedule_date);

CREATE INDEX function_variant_index_hash ON function_variants USING hash(function_variant);

CREATE INDEX machine_supplies_datetime_index_btree ON machine_supplies USING
    btree(machine_supplies_datetime);
CREATE INDEX machine_condition_datetime_index_btree ON machine_conditions USING
    btree(machine_condition_datetime);
```

При процессе авторизации пользователь вводит свой логин, либо email по этой причине при авторизации происходит запрос к базе данных с поиском пользователя по логину или почте, поэтому для почты и логина были созданы индексы. Используются индексы на основе хеш-функций, так как требуется только поиск по соответствию к конкретному значению.

Использование индексов на основе хеш-функций является наиболее оптимальным, в виду линейного алгоритмического времени поиска значения и меньшего занимаемого объема памяти, чем при использовании взвешенного дерева.

Требуется получение выборки заказов по их типу или статусу. Например, для того чтобы показывать пользователю, только заказы на печать, или на сканирование, либо оплаченные или не оплаченные заказы.

Для расписания работы вендинговой точки требуется поиск по дате, или дню недели, чтобы определить работает ли в день когда пользователь зашел на сайт вендинговая точка и какой ее график работы.

Для запасов обычно будет требоваться найти последние по времени записи, по этой причине используются индексы на основе взвешенного дерева, так как оно позволяет почти моментально найти максимальное значение, то есть последнюю дату в этом случае.

Перечисленные индексы зачастую будут использоваться в качестве частей запросов, либо в виде подзапросов совместно с первичными ключами. Перечислять все запросы по первичным ключам в виду тривиальности нецелесообразно, так как ко всем сущностям в базе данных будут часто применяться подобные запросы.

Вывод

В ходе выполнения данного этапа курсовой работы была реализована даталогическая модель в реляционной СУБД PostgreSQL. Целостность данных была обеспечена при помощи средств языка DDL, и также были добавлены триггеры для обеспечения комплексной целостности. Также был проведен анализ созданной базы данных и на его основе для оптимизации запросов, были созданы индексы.